A non-intrusive numerical method for determining the most dangerous unstable modes of discretized multiphase/multimaterial systems. *

Okojunu, Abel
Dept. of math, FL State Univ.

Nair, Unnikrishnan Sasidharan
Dept. of Mech. Engr., FL State Univ.

Sussman, Mark
Dept. of math, FL State Univ.

October 11, 2026

ABSTRACT: A linear stability analysis technique is presented for finding the most unstable modes for a discretized multiphase/multi material system. Instead of explicitly linearizing a discretized system of differential equations and then investigating the spectrum of the linearized equations, it is proposed to do a "matrix-free" DMD method approach. In the proposed linear stability analysis approach, the largest (in magnitude) eigenvalue of the linearized discretized equations is found. The new DMD method approach determines the action of the linear operator, associated with the linearized equations, on a given eigenfunction by way of strategically perturbing a base state and then applying the original, unmodified, multiphase flow algorithm. The multiphase flow algorithm is the coupled level set and "continuous MOF" method for multiphase/multimaterial problems. Since the underlying multiphase flow solver is an Eulerian interface capturing approach, solutions with complex topology can be recovered automatically without interface "surgery." Furthermore, since the proposed linear stability algorithm does not require modification of the underlying flow solver, then it is possible to find unstable modes for complex topology problems. Examples are included for the Rayleigh capillary instability problem in the axisymmetric *R-Z* coordinate system.

*this material is based upon work supported by National Aeronautics and Space Administration under grant number 80NSSC20K0352

1 Introduction

The research question at hand is "can one do a linear stability analysis of complex multiphase flows?" The classical approaches for these kinds of problems become very complicated as the "linearize then discretize" approach becomes unmanagible for complex liquid/gas interface structure. We refer to The following are some earlier works for the linear stability analysis of multiphase flows. In [4], the classical approach of "linearize then discretize" was presented for analyzing stability of an elliptic liquid jet in air. In [3], the classical approach of "linearize then discretize" was presented for analyzing stability of a liquid jet in weak crossflowing air. In [21], the classical approach of "linearize then discretize" was presented for analyzing stability of surface waves of a liquid jet injected in transverse gas flow with different angles.

The strategy taken in order to answer the research question is to test a non-intrusive linear stability analysis algorithm[33] on our in-house multiphase flow code. The in-house code uses dynamic adaptive mesh refinement. We note a recent related article that efficiently implements proper orthogonal decomposition to an existing adaptive mesh refinement code: [27].

2 Mathematical Model

It is assumed that the flow is incompressible.
In the liquid:

$$\rho^L \frac{Du^L}{Dt} = \nabla \cdot (2\mu D^L - p^L I) \tag{1}$$

$$\rho^L c_v^L \frac{DT^L}{Dt} = \nabla \cdot k \nabla T^L \tag{2}$$

$$\nabla \cdot u^L = 0 \tag{3}$$

In the gas:

$$\rho^G \frac{Du^G}{Dt} = \nabla \cdot (2\mu D^G - p^L I) \tag{4}$$

$$\rho^G c_v^G \frac{DT^G}{Dt} = \nabla \cdot k \nabla T^G \tag{5}$$

$$\nabla \cdot u^G = 0 \tag{6}$$

Jump conditions on the interface:

$$[n \cdot \tau \cdot n] = \sigma \kappa \tag{7}$$

$$[u] = 0 \tag{8}$$

$$[T] = 0 \tag{9}$$

$$[k\nabla T \cdot n] = 0 \tag{10}$$

$$\tau = 2\mu D - pI \tag{11}$$

κ is curvature,

$$\kappa = \nabla \cdot n \tag{12}$$

n is the interface normal, written in terms of the level set function ϕ:

$$n = \frac{\nabla \phi}{|\nabla \phi|} \tag{13}$$

The liquid occupies the region where $\phi > 0$ and the gas occupies the region where $\phi < 0$ and ϕ satisfies the following equation:

$$\phi_t + u \cdot \nabla \phi = 0 \tag{14}$$

The overall system can be written as,

$$\frac{dS}{dt} = N(S) \tag{15}$$

$$S = \begin{pmatrix} \phi \\ u \\ T \end{pmatrix} \tag{16}$$

After discretization of the multiphase flow equation, one can write:

$$S^{n+1} = S^n + \Delta t \tilde{N}(S^n) \tag{17}$$

The proposed algorithm, "discretize then differentiate," for linear stability analysis of two phase flows is as follows:

1. compute (17) until a desired "steady state" is reached. We call the steady state $S^{steady,(0)}$. $\bar{Q}$ in [33].

2. Using $S^{steady,(0)}$ as initial conditions, we solve (17) for M time steps, with a fixed Δt. We call the resulting new state $S_{alt}^{steady,(0)}$; this state can be also classified as the "no perturbation state." In other words, $S_{alt}^{steady,(0)}$ is the state that one obtains if the applied perturbation is null. $\bar{Q} + \Delta\bar{Q}$

in [33]. In mathematical form, we define the overall nonlinear operator encompassing the M time steps as:

$$N^{(M)} \equiv (I + \Delta t \tilde{N})^M \tag{18}$$

$N^{(M)}$ is the "M" time step discrete solution operator. Then we have:

$$S_{alt}^{steady,(0)} = N^{(M)}(S^{steady,(0)}) \tag{19}$$

3. $k_{DMD} = 0$

4. Construct default perturbation(s):

$$\delta u^{(k_{DMD})} \tag{20}$$

$$\delta T^{(k_{DMD})} \tag{21}$$

$$\delta \phi^{(k_{DMD})} \tag{22}$$

Q' in [33].

5. (a) normalize the velocity perturbation so that the maximum value is one. The normalized velocity perturbation can be written as:

$$\delta u^{(k_{DMD})} = \frac{\delta u^{(k_{DMD})}}{\alpha_u^{(k_{DMD})}} \tag{23}$$

(b) Multiply the normalized velocity field by a representative perturbation factor:

$$\delta u^{(k_{DMD})} = \epsilon_u \delta u^{(k_{DMD})} \tag{24}$$

6. (a) normalize the temperature perturbation so that the maximum value is one. The normalized temperature perturbation is called:

$$\delta T^{(k_{DMD})} = \frac{\delta T^{(k_{DMD})}}{\alpha_T^{(k_{DMD})}} \tag{25}$$

(b) Multiply the normalized temperature field by a representative perturbation factor:

$$\delta T^{(k_{DMD})} = \frac{\delta T^{(k_{DMD})}}{\alpha_T^{(k_{DMD})}} \tag{26}$$

7. (a) Make sure the input levelset perturbation is zero outside of a user prescribed narrow band thickness R_{narrow} with respect to the unperturbed

level set function. In otherwords, $\delta\phi^{(k_{DMD})} = 0$ when $|\phi^{steady,(0)}| > R_{narrow}$. Define the following interface normalization factor:

$$\beta = \max_{|\phi^{steady,(0)}|<\Delta x} |\delta\phi^{(k_{DMD})}|$$

Multiply $\delta\phi^{(k_{DMD})}$ by $1/\beta$; the normalized level set perturbation is

$$\delta\phi^{(k_{DMD})} = \frac{\delta\phi^{(k_{DMD})}}{\beta^{(k_{DMD})}} \tag{27}$$

(b) Multiply the normalized levelset field by a representative perturbation factor:

$$\delta\phi^{(k_{DMD})} = \sigma\Delta x\delta\phi^{(k_{DMD})} \tag{28}$$

8. Add the perturbations to the base state $S^{steady,(0)}$ (velocity, temperature, and level set function):

$$u^{perturb,(0)} = u^{steady,(0)} + \delta u^{(k_{DMD})} \tag{29}$$

$$T^{perturb,(0)} = T^{steady,(0)} + \delta T^{(k_{DMD})} \tag{30}$$

$$\phi^{perturb,(0)} = \phi^{steady,(0)} + \delta\phi^{(k_{DMD})} \tag{31}$$

note: we can write the overall perturbation in the following form:

$$\delta S^{(k_{DMD})} = \begin{pmatrix} \delta u^{(k_{DMD})} \\ \delta T^{(k_{DMD})} \\ \delta\phi^{(k_{DMD})} \end{pmatrix} \tag{32}$$

$$S^{perturb,(0)} = S^{steady,(0)} + \delta S^{(k_{DMD})} \tag{33}$$

$\bar{Q} + Q'^n$ in [33] ($n = k_{DMD}$).

9. Solve,

$$S^{perturb,(n+1)} = S^{perturb,(n)} + \Delta t\tilde{N}(S^{perturb,(n)}) \tag{34}$$

for M time steps, with a fixed Δt.

10. Update the perturbation prediction:

$$\delta u^{(k_{DMD}+1)} = u^{perturb,M} - u^{(0)}_{alt} \tag{35}$$

$$\delta T^{(k_{DMD}+1)} = T^{perturb,M} - T^{(0)}_{alt} \tag{36}$$

$$\delta\phi^{(k_{DMD}+1)} = \phi^{perturb,M} - \phi^{(0)}_{alt} \tag{37}$$

$$\bar{Q} + Q'^n + \Delta\bar{Q} + \Delta Q'^n - (\bar{Q} + \Delta\bar{Q}) \equiv Q'^{(n+1)} \tag{38}$$

in [33]

11. $k_{DMD} = k_{DMD} + 1$ (n in [33])

12. Go back to step 5.

Remark: the steps 5 through 12 represents an approximation for the following: $x^{k+1} = Ax^k$. So that a Krylov subspace is being generated. To see this we define the overall nonlinear operator encompassing the M time steps as:

$$N^{(M)} \equiv (I + \Delta t \tilde{N})^M \tag{39}$$

$N^{(M)}$ is the "M" time step discrete solution operator.
So that, in this compact notation, step (10) is,

$$\delta S^{(k_{DMD}+1)} = N^{(M)}(S^{steady,(0)} + \delta S^{(k_{DMD})}) - N^{(M)}(S^{steady,(0)}) \tag{40}$$

Note:

$$S_{alt}^{steady,(0)} = N^{(M)}(S^{steady,(0)}) \tag{41}$$

Let,

$$A \equiv \frac{\partial N^{(M)}(S^{steady,(0)})}{\partial S} \tag{42}$$

Then using Taylor's theorem,

$$\delta S^{(k_{DMD}+1)} \approx A\delta S^{(k_{DMD})} \approx A^2 \delta S^{(k_{DMD}-1)} \approx \ldots \approx A^{k_{DMD}+1}\delta S^{(0)} \tag{43}$$

Ranjan et al[33] (paper now included in this repository) recommends to use the DMD method by Schmid[36] (this paper also now included in this repository).

Another paper of interest is "Constrained Dynamic Mode Decomposition"; see [22]. Also this paper is now included in this repository.

2.1 Algorithm for approximating eigenvalues and eigenfunction from a Krylov subspace

In this section only, we let M be the dimension of eigenvectors for which we seek and S an $N-1 \times N-1$ matrix whose eigenvalues approximate that of A. A is an $M \times M$ matrix.
In [36], they first assume that there is a linear mapping A that connects the flow field $\vec{v}_i$ to the subsequent flow field $\vec{v}_{i+1}$, in other words,

$$\vec{v}_{i+1} = A\vec{v}_i. \tag{44}$$

It is assumed that A is approximately independent of time over a given full sampling interval $0 \leq t \leq (N-1)\Delta t$. A Krylov sequence can be generated, given our assumptions, as follows:

$$V_1^N = \left\{v_1, Av_1, A^2v_1, \ldots, A^{N-1}v_1\right\}. \tag{45}$$

The Krylov subspace V_1^N has N vectors; each vector $M \times 1$. This implies that V_1^N can be represented as a $M \times N$ matrix.
It is assumed that $N << M$.
The strategy is to pick N large enough so that the vectors that make up V_1^N are as close to linearly dependent as possible. Then one has,

$$\vec{v}_N = a_1\vec{v}_1 + a_2\vec{v}_2 + \ldots + a_{N-1}\vec{v}_{N-1} + \vec{r} \tag{46}$$

In matrix form (see (2.5) of [36]), one has

$$\vec{v}_N = V_1^{N-1}\vec{a} + \vec{r}. \tag{47}$$

$\vec{a}$ is defined as

$$\vec{a}^T = (a_1, a_2, \ldots, a_{N-1}) \tag{48}$$

Next, in [36] equation (2.6), one has

$$AV_1^{N-1} = V_2^N = \tag{49}$$
$$\left\{\vec{v}_2, \vec{v}_3, \vec{v}_4, \ldots, V_1^{N-1}\vec{a}\right\} + \vec{r}\vec{e}_{N-1}^T =$$
$$V_1^{N-1}S + \vec{r}\vec{e}_{N-1}^T \tag{50}$$

where ([36] equation (2.8)),

$$S = \begin{pmatrix} 0 & & & & a_1 \\ 1 & 0 & & & a_2 \\ & \ddots & \ddots & & \vdots \\ & & 1 & 0 & a_{N-2} \\ & & & 1 & a_{N-1} \end{pmatrix} \tag{51}$$

To summarize, one has,

$$AV_1^{N-1} = V_1^{N-1}S + \vec{r}\vec{e}_{N-1}^T. \tag{52}$$

S is an $N-1 \times N-1$ matrix.
Suppose $\vec{r} = 0$, then (from (47))

$$\vec{v}_N = V_1^{N-1}\vec{a} \tag{53}$$

Let,

$$U\Sigma W^* = V_1^{N-1} \qquad \text{(SVD sorted from highest to lowest singular values)} \tag{54}$$

$$\vec{v}_N = U\Sigma W^*\vec{a} \tag{55}$$

From (55):

$$\vec{a} = W\Sigma^\dagger U^*\vec{v}_N \tag{56}$$

V_1^{N-1} is a $M \times N-1$ matrix. U is an $M \times M$ matrix. Σ is a $M \times N-1$ matrix. W is a $N-1 \times N-1$ matrix.
IMPORTANT REMARK: There should be a way, using the "Implicitly restarted Arnoldi method"[23], in order to find the top eigenvalues and eigenvectors of V_1^{N-1} using just matrix vector multiplies where the matrix is V_1^{N-1}.
ANOTHER IMPORTANT REMARK: one can solve

$$\vec{v}_N = V_1^{N-1}\vec{a} \tag{57}$$

for $\vec{a}$ by framing the problem as a least squares minimization problem.
So now, given that $\vec{a}$ is determined, we can find S using (51). Then, using (52) and ignoring $\vec{r}$, is to observe that,

$$AU\Sigma W^* = U\Sigma W^* S \tag{58}$$

$$U^*AU\Sigma W^* = \Sigma W^* S \tag{59}$$

$$U^*AU\Sigma = \Sigma W^* SW \tag{60}$$

$$U^*AU = \Sigma W^* SW\Sigma^\dagger \tag{61}$$

The matrix on the right of (61) is an $M \times M$ matrix which is large! But the dimensions of S is $N-1 \times N-1$. So we find the eigenvalues and eigenvectors of S:

$$S = X\Gamma X^{-1} \qquad \text{(sorted from highest to lowest magnitude eigenvalues)} \tag{62}$$

Using (62) and (61), one has

$$A \approx P\Gamma P^\dagger \qquad P = U\Sigma W^* X = V_1^{N-1}X \tag{63}$$

P is an $M \times N-1$ matrix.

So now using an "approximate" similarity transpose, we have that the first $N-1$ eigenvectors of A are approximately the columns of P and the dominant eigenvalues of A are the diagonal entries of Γ.

2.2 Equivalency of the above analysis to the least squares approach and a new generalization

In re-considering equations (55) through (57), one can cast the problem as a least squares problem without ever explicitly finding the $M \times M$ matrix U. In order to simplify the notation, define:

$$B \equiv V_1^{N-1}. \tag{64}$$

The least squares problem is:

$$\vec{a} = \text{argmin} J(\vec{a}) = \text{argmin}_{\vec{a}} \sum_{i=1}^{M} \left((\sum_{j=1}^{N-1} B_{ij} a_j) - v_{Ni} \right)^2 w_i \tag{65}$$

The system of equations corresponding to,

$$\frac{\partial J}{\partial a_k} = 0 \quad k = 1, \ldots, N-1, \tag{66}$$

is,

$$B^T W B \vec{a} = B^T W \vec{v}_N. \tag{67}$$

W is a $M \times M$ diagonal matrix with the following components:

$$W_{ij} = \begin{cases} w_i & \text{if } i = j \\ 0 & \text{otherwise} \end{cases} \tag{68}$$

The components of $B^T W B$ and $B^T W \vec{v}_N$ are weighted inner products of the Krylov subspace vectors amongst themselves:

$$(B^T W B)_{ij} = (\vec{v}_i, \vec{v}_j)_{\vec{w}} \quad i, j = 1, \ldots, N-1 \tag{69}$$

$$(B^T W \vec{v}_N)_i = (\vec{v}_i, \vec{v}_N)_{\vec{w}} \quad i = 1, \ldots, N-1 \tag{70}$$

Remarks:

1. Solving (67) for $\vec{a}$, by finding the pseudo inverse (Moore-Penrose inverse) of $B^T W B$, is equivalent to (56) when W is the identity.

2. The weights should be chosen according to the positioning of the zero level set of "ϕ^N". For interfacial flows controlled by the interface, the weights corresponding to the level set function variable should dominate over the weights corresponding to the velocity variables.

3 Background - deforming boundary problems in fluid mechanics

In the Introduction, we briefly demonstrate the benefits of our new method over the current state-of-the-art numerical methods for multi-phase flows in which surface tension forces are important. In this section, we give an overview of numerical methods for deforming boundary problems in computational fluid dynamics in general, and show where our new method fits in.

For many deforming boundary problems in fluid dynamics, "shock capturing" numerical methods are sufficient[15, 9, 45, 16, 38, 35]. Shock capturing

methods have limitations. According to "Godunov's theorem"[14], linear one-step second order accurate numerical methods for $\phi_t + c\phi_x = 0$ cannot be monotonicity preserving unless $|c|\Delta t/\Delta x$ is an integer. The concept of "monotonicity preserving" methods and its association to TVD (Total Variational Diminishing) methods was first drawn by Harten[16] who proved that (a) A monotone scheme is TVD and (b) a TVD scheme is monotonicity preserving. When Harten's theory is taken together with Godunov's theorem, one can see that "linear" second order TVD methods only exist under specialized situations. Harten's and Godunov's theory are manifested by the fact that stable shock capturing methods will invariably smear discontinuities over time.

In order to overcome the limitations of shock capturing methods, researchers have developed (i) front tracking methods[13, 43], (ii) front capturing methods[26, 30, 41, 17, 7, 1, 2, 29, 32, 18], and (iii) shock fitting methods[34].

The above methods for overcoming the the time dependent "interface smearing" problem of the shock capturing methods, have been further improved by way of hybridization: the particle level set method[11], the hybrid front tracking and level set method[37], the coupled level set and volume of fluid method[40], and the coupled level set and moment of fluid method[6, 19].

The numerical method that we introduce in this article falls in the latter "hybrid" category in which we introduce the hybrid level set and Continuous Moment of Fluid (CMOF) method for multiphase/multimaterial flows. The level set method and the continuous moment-of-fluid method are hybridized as follows: (a) the smooth level set distance function representation is used to provide an initial guess for the CMOF slope reconstruction step (see Figure **??** and Section 5.2), and (b) the level set distance functions are in turn replaced by the exact signed distance to the continuous moment-of-fluid reconstructed interface (see Figure **??** and Section 5.3). The level set and CMOF representations are synchronized every time step.

4 Mathematical model

We simulate the flow of a multiphase system consisting of M_{fluid} fluid (deforming) materials and M_{rigid} non-deforming materials. The M_{fluid} materials tessellate the computational domain and in the case that a fluid material m_{fluid} coincides with a rigid material, m_{rigid}, the rigid materials' governing equations take precedence. Otherwise, in regions where fluid materials and rigid materials do not overlap, the fluid materials are governed by the incompressible Navier-Stokes equations of immiscible flows. We refer the reader to Figures **??**, **??**, and **??** for illustrations which distinguish between deforming (tessellating) materials and non-deforming (rigid) materials.

- ▷ **Material domain and interface:** Mathematically, the domain of rigid *material* m_{rigid} is the region in which $\phi_{m_{rigid}} > 0$:

$$\phi_{m_{rigid}}(\mathbf{x}, t) = \begin{cases} > 0 & \mathbf{x} \in \text{material } m_{rigid}, \\ \leq 0 & \text{otherwise}, \end{cases} \tag{71}$$

$\boldsymbol{x}$ is the position vector in space and t is time. The domain of *material* m_{fluid} is the region in which $\phi_{m_{fluid}} > 0$ and $\phi_{m_{rigid}} < 0$:

$$\phi_{m_{fluid}}(\mathbf{x}, t) = \begin{cases} > 0 & \mathbf{x} \in \text{material } m_{fluid} \cup m_{fluid,ghost}, \\ \leq 0 & \text{otherwise}, \end{cases} \tag{72}$$

The *interface* level set, $\phi_{m1,m2}$ represents the interface between materials $m1$ and $m2$

$$\phi_{m1,m2}(\mathbf{x}, t) = \begin{cases} > 0 & \mathbf{x} \in \text{material } m1, \\ < 0 & \mathbf{x} \in \text{material } m2, \\ = 0 & \mathbf{x} \text{ along } (m1, m2) \text{ interface}. \end{cases} \tag{73}$$

The normal and curvature defined based on these level set functions are:

$$\mathbf{n}_{m1,m2} = \frac{\nabla \phi_{m1,m2}}{|\nabla \phi_{m1,m2}|}, \quad \kappa_{m1,m2} = \nabla \cdot \frac{\nabla \phi_{m1,m2}}{|\nabla \phi_{m1,m2}|}. \tag{74}$$

▷ **Conservation of mass:** We assume that each fluid material, m_{fluid}, is incompressible, so that the velocity field $\boldsymbol{u} = (u, v, w)$ is divergence free within the bulk of each fluid material:

$$\nabla \cdot \boldsymbol{u} = 0. \tag{75}$$

In order to account for phase change or otherwise sources and sinks of mass, we have the following conditions on $\nabla \cdot \boldsymbol{u}$:

$$\nabla \cdot \boldsymbol{u} = \sum_{\text{sources}} \frac{\dot{m}_{\text{source}}}{\rho_{\text{source}}} \delta(\phi_{m_{\text{source}}}) - \sum_{\text{sinks}} \frac{\dot{m}_{\text{sink}}}{\rho_{\text{sink}}} \delta(\phi_{m_{\text{sink}}})$$

$H(\phi)$ is the Heaviside function which is defined as,

$$H(\phi) = \begin{cases} 1 & \phi > 0 \\ 0 & \phi \leq 0 \end{cases}$$

$\delta(\phi)$ is the Dirac Delta function,

$$\delta(\phi) = H'(\phi).$$

For boiling examples, $\dot{m}$ is the mass flux of boiling liquid across the liquid/vapor interface,

$$\dot{m} = \frac{k_l \nabla T_l \cdot \boldsymbol{n}_{l,v} - k_v \nabla T_v \cdot \boldsymbol{n}_{l,v}}{L},$$

where k_l and k_v are the thermal conductivities in the liquid and ambient vapor regions respectively, ρ_l and ρ_v are the densities in the liquid and ambient vapor regions respectively, L is the latent heat of vaporization, and $\boldsymbol{n}_{l,v}$ is the interface normal vector pointing from the ambient vapor region into the liquid,

$$\boldsymbol{n}_{l,v} = \frac{\nabla \phi_{l,v}}{|\nabla \phi_{l,v}|}.$$

▷ **Conservation of momentum:** The conservation of momentum for each material in its domain is given by

$$(\boldsymbol{u}\rho_m)_t + \nabla \cdot (\boldsymbol{u} \otimes \boldsymbol{u}\rho_m + p_m \mathbb{I}) = \nabla \cdot (2\mu_m \mathbb{D}) + \rho_m \boldsymbol{g}(1 - \alpha_m(T_m - T_{0m})) \quad \text{if} \quad \phi_m(\boldsymbol{x}, t) > 0, \tag{76}$$

where p_m, T_m, α_m, and μ_m are pressure, temperature, coefficient of thermal expansion, and viscosity of material m respectively, $\mathbf{g}$ is the gravitational acceleration vector, and $\mathbb{D} = (\nabla \boldsymbol{u} + (\nabla \boldsymbol{u})^T)/2$ is the rate of deformation tensor.

▷ **Conservation of energy:** The conservation of energy for each material in its domain is given by

$$(\rho_m C_{p,m} T_m)_t + \nabla \cdot (\boldsymbol{u}\rho_m C_{p,m} T_m) = \nabla \cdot (k_m \nabla T_m) \quad \text{if} \quad \phi_m(\boldsymbol{x}, t) > 0, \tag{77}$$

where $C_{p,m}$ and k_m are heat capacity and thermal conductivity of material m respectively, and T_m is the temperature.

▷ **Interfacial jump condition:** Here, we write out the general equations for a deforming interface changing phase. We define m_s and m_d as the material id's associated with a "source" material (e.g. boiling liquid or freezing liquid) and "destination" material (e.g. vapor from boiling or ice from freezing) respectively. The location of the interface separating a material m_s region from a material m_d region is governed by the level set equation,

$$\phi_{m_s,m_d,t} + \boldsymbol{u}_{m_s} \cdot \nabla \phi_{m_s,m_d,t} = -\frac{\dot{m}}{\rho_s} |\nabla \phi_{m_s,m_d,t}| \tag{78}$$

$$\phi_{m_d,m_s,t} + \boldsymbol{u}_{m_s} \cdot \nabla \phi_{m_d,m_s,t} = \frac{\dot{m}}{\rho_s} |\nabla \phi_{m_d,m_s,t}| \tag{79}$$

An equivalent expression for the level set governing equations is:

$$\phi_{m_s,m_d,t} + \boldsymbol{u}_{m_d} \cdot \nabla \phi_{m_s,m_d,t} = -\frac{\dot{m}}{\rho_d} |\nabla \phi_{m_s,m_d,t}| \tag{80}$$

$$\phi_{m_d,m_s,t} + \boldsymbol{u}_{m_d} \cdot \nabla \phi_{m_d,m_s,t} = \frac{\dot{m}}{\rho_d} |\nabla \phi_{m_d,m_s,t}| \tag{81}$$

The interface jump conditions, between two materials m_s and m_d, for the velocity, pressure, and temperature, respectively, are,

$$\boldsymbol{u}_{m_s} \cdot \boldsymbol{n}_{m_s,m_d} - \boldsymbol{u}_{m_d} \cdot \boldsymbol{n}_{m_s,m_d} = \dot{m}\left(\frac{1}{\rho_{m_d}} - \frac{1}{\rho_{m_s}}\right),$$

$$(p_{m_s}\mathbb{I} - p_{m_d}\mathbb{I}).\boldsymbol{n}_{m_s,m_d} = -\sigma_{m_s,m_d}\kappa_{m_s,m_d}\boldsymbol{n}_{m_s,m_d} + (2\mu_{m_s}\mathbb{D}_{m_s} - 2\mu_{m_d}\mathbb{D}_{m_d}).\boldsymbol{n}_{m_s,m_d},$$

$$T_{m_s} = T_{m_d}. \tag{82}$$

κ_{m_s,m_d} is the interface curvature and is defined as,

$$\kappa_{m_s,m_d} = \nabla \cdot \frac{\nabla \phi_{m1,m2}}{|\nabla \phi_{m1,m2}|}.$$

At a triple point junction a three-phase equilibrium known as the Neumann's triangle[10] determines the angles (see Figure **??**.a):

$$\frac{\sin(\theta_1)}{\sigma_{23}} = \frac{\sin(\theta_2)}{\sigma_{13}} = \frac{\sin(\theta_3)}{\sigma_{12}}. \tag{83}$$

5 Numerical methods and algorithms

We describe our numerical method for the 2D uniform rectangular Cartesian grid case. In the problem domain, a computational cell, $\Omega_{i,j}$, is defined as,

$$\Omega_{i,j} = \left\{ \boldsymbol{x} : x \in \left[x_i - \frac{\Delta x}{2}, x_i + \frac{\Delta x}{2} \right], \right. \tag{84}$$

$$\left. y \in \left[y_i - \frac{\Delta y}{2}, y_i + \frac{\Delta y}{2} \right] \right\} \tag{85}$$

where $\boldsymbol{x}_{i,j} = \{x_i, y_j\}$ is the center of the cell $\Omega_{i,j}$. The domain of material m in a cell at time t^n is denoted by $\Omega^n_{m,i,j}$, and the zeroth and first order moments of the mth material distribution, corresponding to the volume fraction and centroid position, are defined as,

$$F^n_{m,i,j} = \frac{\int_{\Omega^n_{m,i,j}} \mathrm{d}\Omega}{V_{i,j}}, \qquad \boldsymbol{x}^{c,n}_{m,i,j} = \frac{\int_{\Omega^n_{m,i,j}} \boldsymbol{x}\ \mathrm{d}\Omega}{V^n_{i,j,m}}, \tag{86}$$

where the computational cell volume is $V_{i,j} = \int_{\Omega_{i,j}} \mathrm{d}\Omega$, and volume of the portion for material m in a computational cell is $V^n_{i,j,m} = \int_{\Omega^n_{m,i,j}} \mathrm{d}\Omega$. The discretization for a typical point around the triple point is shown in Figure **??**.

5.1 method overview: Staggered grid Projection Method

Referring to Figure **??** and also [31, 44], we discretize the velocity on the marker-and-cell (MAC) grid and we discretize pressure, temperature, level set function(s), volume fraction(s), and centroid(s) at the center of grid cells.

An outline of our operator split method is as follows ([44]):

▷ **Calculate the time step $\Delta t = t^{n+1} - t^n$:**

$$\Delta t_1 = \min_{d=1,\ldots,\mathrm{D}} \frac{\min \Delta x_d}{2D \max |u^n_d|} \qquad \mathrm{D} = 2 \text{ or } 3$$

$$\Delta t_2 = \min_{d=1,\ldots,\mathrm{D}} \frac{\min \Delta x_d}{D \max |u_d^{\text{phase change},n-1}|}$$

$$\Delta t_3 = \min_{d=1,\ldots,D} \min_{m1,m2=1,\ldots,M} \Delta x_d^{3/2} \sqrt{\frac{\rho_{m1} + \rho_{m2}}{2\pi\sigma_{m1,m2}}} \tag{87}$$

$$\Delta t = \mathrm{CFL} \min(\Delta t_1, \Delta t_2, \Delta t_3) \qquad \mathrm{CFL} = 1/2$$

▷ **Cell Integrated Semi-Lagrangian (CISL) MAC velocity advection (see[31]):**

$$(\rho \boldsymbol{u})_t + \nabla \cdot (\rho \boldsymbol{u}\boldsymbol{u}) = 0, \tag{88}$$

▷ **CISL temperature advection (liquid and gas materials, see [44]):**

$$(\rho_m C_{p,m} T_m)_t + \nabla \cdot (\rho_m C_{p,m} \boldsymbol{u} T_m) = 0, \quad m = 1, \ldots, M \tag{89}$$

▷ **CISL level set advection:**

$$\phi_{m,t} + \boldsymbol{u} \cdot \nabla \phi_m = 0, \quad m = 1, \ldots, M$$

▷ **CISL advection of the Volume fractions and Centroids (see [19, 20, 24, 31, 44]):**

$$F_{m,t} + \nabla \cdot (\boldsymbol{u} F_m) = 0, \quad m = 1, \ldots, M \tag{90}$$

▷ **Coupling the level set functions to the volume fractions and Centroids:** The (continuous) moment-of-fluid reconstructed (see Section 5.2) slope initial guess uses the level set functions, and the level set functions are in turn replaced by the exact signed distance (see Section 5.3) to the continuous moment-of-fluid reconstructed interface.

▷ **Phase change velocity from material m_s (source) to material m_d (destination) (see [44] and [25]):**

$$\boldsymbol{u}^{\text{phase change}} = -\frac{\dot{m}}{\rho_{m_d}} \boldsymbol{n}_{m_d}, \tag{91}$$

▷ **Phase change: Level Set Advection:**

$$\phi_{m,t} + \boldsymbol{u}^{\text{phase change}} \cdot \nabla \phi_m = 0, \quad m = m_s \text{ or } m_d.$$

▷ **Phase change: unsplit CISL advection of the Volume fractions and Centroids (see [44, 25]):**

$$F_{m,t} + \nabla \cdot (\boldsymbol{u}^{\text{phase change}} F_m) = 0, \quad m = m_s \text{ or } m_d.$$

▷ **Phase change: Coupling the level set functions to the volume fractions and Centroids:** The (continuous) moment-of-fluid reconstructed (see Section 5.2) slope initial guess uses the level set functions, and the level set functions are in turn replaced by the exact signed distance (see Section 5.3) to the continuous moment-of-fluid reconstructed interface.

▷ **Phase change: Mass source redistribution:** Redistribute $\dot{m}$ to the source material, m_s, side (e.g. liquid if boiling or freezing) [44].

▷ **Thermal diffusion (see [44] for algorithmic details):**

$$\frac{(\rho C_{p,m})^{\text{mix},n+1}}{\Delta t^{\text{swept}}}(T_m^{n+1} - T_m^{\text{advection}}) = \nabla \cdot (k_m \nabla T_m^{n+1}) \tag{92}$$

▷ **Viscosity (see [31] for algorithmic details):**

$$\frac{\rho^{\text{mix},n+1}}{\Delta t}(\boldsymbol{u}^* - \boldsymbol{u}^{\text{advection}}) = \nabla \cdot (2\mu \mathbb{D}^*) - \rho^{n+1}(\alpha^{n+1}(T^{n+1} - T_0))\boldsymbol{g} \tag{93}$$

▷ **Pressure Gradient (liquid and vapor regions) (see [20, 24, 31, 44] for algorithmic details):**

$$\frac{\boldsymbol{u}^{n+1} - \boldsymbol{u}^*}{\Delta t} = -\frac{\nabla p^{n+1}}{\rho^{\text{MAC,mix},n+1}} + \boldsymbol{g} - \frac{\sum_{m=1}^{M} \gamma_m \kappa_m \nabla H(\phi_m)}{\rho^{\text{MAC,mix},n+1}} \tag{94}$$

$$\nabla \cdot \boldsymbol{u}^{n+1} = \sum_{\text{sources}} \frac{\dot{m}_{\text{source}}}{\rho_{\text{source}}} \delta(\phi_{m_{\text{source}}}) - \sum_{\text{sinks}} \frac{\dot{m}_{\text{sink}}}{\rho_{\text{sink}}} \delta(\phi_{m_{\text{sink}}})$$

For the case in which two materials, $m1$ and $m2$, are present in a given 3×3 stencil,

$$\gamma_{m1} = \gamma_{m2} = \frac{\sigma_{m1,m2}}{2}$$

and for the case when three materials, $m1$, $m2$, $m3$, are present in a given 3×3 stencil,

$$\gamma_{m1} = \frac{\sigma_{m1,m2} + \sigma_{m1,m3} - \sigma_{m2,m3}}{2}$$

$$\gamma_{m2} = \frac{\sigma_{m1,m2} + \sigma_{m2,m3} - \sigma_{m1,m3}}{2}$$

$$\gamma_{m3} = \frac{\sigma_{m1,m3} + \sigma_{m2,m3} - \sigma_{m1,m2}}{2}$$

Remarks:

1. For step 2 "phase change" above, we use MOF interface reconstruction. Our rationale is that (1) the phase change velocity $\boldsymbol{u}^{\text{phase change}}$ is not determined from the interface curvature, so that there is no instability risk due to MOF advection here, and (2) MOF is more accurate than CMOF in this scenario.

2. We use the multigrid preconditioned conjugate gradient method [42] to solve the large sparse matrix system that results from discretizing (92). The discretization of the Dirichlet interface temperature condition due to phase change uses the second order method described in [12].

3. We also use the multigrid preconditioned conjugate gradient (MGPCG) method [42] to solve the large sparse matrix system that results from discretizing (94). In order to simulate flows in sealed tanks or flows induced by the sealing of a valve [5] we have developed a general method for enforcing the solvability condition in each "enclosed" region. We identify each "enclosed" region using a shading method developed by [39].

4. In order to preserve mass for phase change problems, we must prescribe a stringent tolerance for the pressure equation (94). In some cases, due to round off error, the MGPCG method might stall. In order to overcome the effect of round-off error, we keep track of a history of the pressure admitting the least residual, and, when the solver has stalled, we restart the MGPCG process using the previous best guess pressure and increasing the number of multigrid relaxation steps by 1.

5.2 MOF and Continuous MOF interface reconstruction methods

The original MOF reconstruction method is local to the cell and uses the reference volume fraction, $F_{\text{ref}} \equiv F^n_{m,i,j}$, and reference centroid, $\boldsymbol{x}^c_{\text{ref}} \equiv \boldsymbol{x}^n_{m,i,j}$ to find the linear (planer) interface reconstruction that has the volume fraction equal to F_{ref}, and has the least amount of error for centroid position. We find the actual volume fraction $F_{\text{act}}(\boldsymbol{n}, b)$ and centroid $\boldsymbol{x}^c_{\text{act}}(\boldsymbol{n}, b)$ for a reconstructed line(plane) with the normal $\boldsymbol{n}$ and intercept b which minimizes

$$E_{\text{MOF}} = \|\boldsymbol{x}^c_{\text{ref}} - \boldsymbol{x}^c_{\text{act}}(\boldsymbol{n}, b)\|_2 \tag{95}$$

while $F_{\text{act}}(\boldsymbol{n}, b) = F_{\text{ref}}$ (see Figure **??**).

Starting from a whole cell and repeating this process while only considering the uncaptured regions from previous steps, we can reconstruct the material interface for each phase in a multimaterial cell (see [24] for algorithm details). This tessellating procedure generates a volume preserving reconstruction at triple-points (see Figure **??**).

The Continuous Moment-of-fluid method (CMOF) employs a similar procedure to find the interface reconstruction in a cell, but uses a different value for the reference cell centroid $\boldsymbol{x}^c_{\text{ref}}$. We define a super cell

$$\Omega^s_{i,j} = \left\{ \boldsymbol{x} : x \in \left[x_i - \frac{3\Delta x}{2}, x_i + \frac{3\Delta x}{2} \right], \right. \tag{96}$$

$$\left. y \in \left[y_i - \frac{3\Delta y}{2}, y_i + \frac{3\Delta y}{2} \right] \right\} \tag{97}$$

with volume fraction and centroid

$$F^{s,n}_{m,i,j} = \frac{\sum_{i,j} F^n_{m,i,j} V_{i,j}}{\sum_{i,j} V_{i,j}}, \quad \boldsymbol{x}^{c,s,n}_{m,i,j} = \frac{\sum_{i,j} F^n_{m,i,j} \boldsymbol{x}^{c,n}_{m,i,j} V_{i,j}}{\sum_{i,j} F^n_{m,i,j} V_{i,j}}. \tag{98}$$

For the CMOF method we find the slope $\boldsymbol{n}$ and intercept b such that $F_{\text{act}}(\boldsymbol{n}, b) = F_{\text{ref}}$, $E_{\text{MOF}} = \|\boldsymbol{x}^c_{\text{ref}} - \boldsymbol{x}^c_{\text{act}}(\boldsymbol{n}, b)\|_2$ is minimized, while $F_{\text{ref}} \equiv F^n_{m,i,j}$, $\boldsymbol{x}^c_{\text{act}}(\boldsymbol{n}, b)$ is measured relative to the super cell $\Omega^s_{i,j}$, and $\boldsymbol{x}^c_{\text{ref}} \equiv \boldsymbol{x}^{c,s,n}_{m,i,j}$. We refer the reader to Figure **??**.

The initial starting guess for finding the optimal MOF or CMOF slope is determined from the optimal choice from the following (described for the 3D case):

1. $(\Theta^{(1)}, \Phi^{(1)}) = \text{slope to angle}(\frac{\boldsymbol{x}_{ref} - \boldsymbol{x}_{uncapt}}{||\boldsymbol{x}_{ref} - \boldsymbol{x}_{uncapt}||})$

2. (first cut only)

$$(\Theta^{(2)}, \Phi^{(2)}) = \text{slope to angle}(\boldsymbol{n}^{(\text{CLSVOF})}). \tag{99}$$

   $\boldsymbol{n}^{(\text{CLSVOF})}$ is derived from the CLSVOF reconstructed slope [40].

3. (first cut only)
   $(\Theta^{(3)}, \Phi^{(3)})$ is determined by way of the following "regression" decision tree [8] machine learning process (see Figure **??**) :

   (a) at the very start of a simulation, randomly create $n_{sample} = 100^3$ sample triplets for planes cutting through a given cell:

$$(\Theta^{sample}, \Phi^{sample}, F^{sample}).$$

   (b) Each sample is associated with $(\Theta^{key}, \Phi^{key}, F^{key})$ which is determined as

$$(\Theta^{key}, \Phi^{key}) = \text{slope to angle}(\frac{\boldsymbol{x}_{ref,sample} - \boldsymbol{x}_{uncapt}}{||\boldsymbol{x}_{ref,sample} - \boldsymbol{x}_{uncapt}||})$$
$$F^{key} = F^{sample}$$

   (c) Each associated triplet, $(\Theta^{key}, \Phi^{key}, F^{key})$ with corresponding classification,

$$(\Theta^{classify}, \Phi^{classify}, F^{classify}) \equiv (\Theta^{sample}, \Phi^{sample}, F^{sample})$$

   is added to the decision tree list.

(d) After all of the sample data has been classified, the decision tree list is split into two branches, determined by the median in the critical splitting direction (Θ, Φ, or F). See Figure **??**. The critical direction maximizes the decrease in the classification variance between the parent and its associated two branches. (each branch having its own mean).

(e) The tree is split again into 4 branches, with the next critical direction determined to maximize the classification variance reduction. Note: the critical direction is the same for each branch on a given level.

(f) the previous step is repeated recursively until each branch in the tree contains just one piece of data. The number of levels in the tree cannot exceed ceiling($\log_2 n_{sample}$).

(g) for the prediction phase, the decision tree is traversed using the key, $(\Theta^{key}, \Phi^{key}, F^{key})$ in which

$$(\Theta^{key}, \Phi^{key}) = \text{slope to angle}(\frac{\boldsymbol{x}_{ref} - \boldsymbol{x}_{uncapt}}{||\boldsymbol{x}_{ref} - \boldsymbol{x}_{uncapt}||}). \qquad (100)$$
$$F^{key} = F_{ref}$$

$(\Theta^{(3)}, \Phi^{(3)})$ is the classification obtained from traversing the decision tree using the key from (100). The maximum number of decisions to be made in order to classify a key not contained in the training data set is ceiling($\log_2 n_{sample}$).

Remarks:

- ▷ In 2D, at the very start of a simulation, one randomly creates $n_{sample} = 100^2$ sample pairs for lines cutting through a given cell: $(\Theta^{sample}, F^{sample})$.
- ▷ In RZ, a 2D decision tree is created corresponding to each discrete value of $r_i = (i + 1/2)\Delta x$.
- ▷ Analytical MOF reconstruction algorithms exist [28], but not analytical CMOF reconstruction algorithms.
- ▷ for the CMOF reconstruction algorithm, $\boldsymbol{x}_{ref}$ and $\boldsymbol{x}_{uncapt}$ are measured relative to the super cell $\Omega^s_{i,j}$ (96). $\boldsymbol{x}_{uncapt}$ is the centroid of the "uncaptured" (i.e. "unreconstructed") region of $\Omega^s_{i,j}$. For the first cut, $\boldsymbol{x}_{uncapt}$ is the centroid of $\Omega^s_{i,j}$ (96).
- ▷ if the super cell, $\Omega^s_{i,j}$, contains at most two materials, then our starting guess is guaranteed to lead to a second order reconstruction, regardless of the number of ensuing optimization iterations. This is because the CLSVOF slope (99) by itself leads to a second order method.
- ▷ We have determined anecdotally that a machine learning sample size of $N^d_{1D} = 100^d$ where d is the dimension of the optimization input data and N_{1D} represents a "sample size per dimension," leads to

errors comparable to iterating to convergence. We have arrived at this sample size by studying the N_{1D} dependence of the symmetric difference error for "Zalesaks" problem (see Table **??**).

5.3 Reconstructing the distance function

Here we describe a distance function reconstruction algorithm that evaluate the exact signed distance function to the reconstructed interface.

Assuming that the reconstructed interface is available on the multimaterial cells, using a MOF or CMOF reconstruction, the algorithm first initialize the distance function to a large number with correct sign. Then in a narrow band the cells tagged for involvement in the redistancing procedure, that is, contributing information to and/or getting updated distance function value through this process. In the next steps, distances are evaluated from the center of an update cell to different possible interfacial points in its neighborhood, and material, ϕ_m, and interface distance functions ,$\phi_{m1,m2}$, are updated consequently.

- ▷ Set all distance functions, ϕ_m and $\phi_{m1,m2}$, to a large negative value.
- ▷ Iterate over all cells.
  - ▷ Using the reconstructed interface, change the sign of ϕ_k and $\phi_{k,m2}$ to positive if material k is occupying the center of a cell.
  - ▷ For a cell, check the neighbors for occupying materials in a $3 \times 3 \times 3$ hypercube stencil around it. Count the materials m present if $F_m > 0.5$ in at least one of the cells in the stencil, or if a material level set changes sign between two different cells in the stencil. If more than one material are present in the stencil, this cell is a *support* cell for the redistancing algorithm. Consequently, tag cells in the $9 \times 9 \times 9$ hypercube around a support cell as *update* cells.
- ▷ Iterate over update cells.
  - ▷ Iterate over the support cells in the $3 \times 3 \times 3$ hypercube around this update cell.
    - ▷ Evaluate distance to the corners, face centers, and cell center in the support cell. This procedure gives the exact distance if cell boundaries are part of the material interface (Figure **??**.a).
    - ▷ Find the normal distance to each interface in the support cell. Update the distance only if the intersection point is within the support cell (Figure **??**.b).
    - ▷ Find the intersection of each pair of the interfaces in the support cell, and evaluate the distance to the triple-point. Check the materials around the intersection for updating the corresponding interface distance functions (Figure **??**.c).

▷ Find the intersection of the interfaces with cell faces, evaluate the distance, and update the corresponding distance functions (Figure **??**.d).

▷ For 3D cases only, find the intersection line of each pair of interfacial planes, and find the intersection of the cell boundaries with the line. Then, evaluate the distance to the intersection points, and update the corresponding distance functions (Figure **??**).

Notes:

- In the redistancing algorithm, after finding the distance to an interface or intersection of interface with cell boundaries or other interfaces the materials around the intersection points are found by testing some points around the intersection to figure out which material they belong to. These points are picked by the combination of interface normal vectors involved in the intersection points and are shown with white diamonds in Figure **??** and Figure **??**.

- Both distance function value and normal vector is updated if a distance measurement is found with smaller magnitude compared to the stored value in ϕ_m and $\phi_{m1,m2}$.

- Applying the algorithm described above, an interface distance function $\phi_{m1,m2}$ is not well defined when the update cell belongs to neither material $m1$ nor material $m2$ (Figure **??**.a). An alternative measurement is used to extend the interface distance functions (Figure **??**.b). For a cell $\{i,j\}$ where $\phi_{m1,i,j} < 0$ and $\phi_{m2,i,j} < 0$ we have:

$$\begin{aligned} \boldsymbol{x}_{m1} &= \boldsymbol{x}_{i,j} - \phi_{m1,i,j}\boldsymbol{n}_{m1,i,j}, && (101)\\ \boldsymbol{x}_{m2} &= \boldsymbol{x}_{i,j} - \phi_{m2,i,j}\boldsymbol{n}_{m2,i,j}, && (102)\\ \boldsymbol{q} &= \boldsymbol{x}_{m1} - \boldsymbol{x}_{m2}. && (103) \end{aligned}$$

$$\begin{cases} \phi_{m1,m2} = |\boldsymbol{q}| & \text{if } |\phi_{m1,i,j}| < |\phi_{m2,i,j}| \\ \phi_{m1,m2} = -|\boldsymbol{q}| & \text{if } |\phi_{m1,i,j}| \geq |\phi_{m2,i,j}| \end{cases} \quad (104)$$

6 Numerical tests and results

References

[1] Ahn, H. T., and Shashkov, M. Multi-material interface reconstruction on generalized polyhedral meshes. *Journal of Computational Physics 226*, 2 (2007), 2096–2132.

[2] Ahn, H. T., and Shashkov, M. Adaptive moment-of-fluid method. *Journal of Computational Physics 228*, 8 (2009), 2792–2821.

[3] Amini, G. Linear stability analysis of a liquid jet in a weak crossflow. *Physics of Fluids 30*, 8 (08 2018), 084105.

[4] Amini, G., Lv, Y., Dolatabadi, A., and Ihme, M. Instability of elliptic liquid jets: Temporal linear stability theory and experimental analysis. *Physics of fluids 26*, 11 (2014).

[5] Arienti, M., and Sussman, M. An embedded level set method for sharp-interface multiphase simulations of diesel injectors. *International Journal of Multiphase Flow 59* (2014), 1–14.

[6] Asuri Mukundan, A., Ménard, T., Brändle de Motta, J. C., and Berlemont, A. A hybrid moment of fluid–level set framework for simulating primary atomization. *Journal of Computational Physics 451* (2022), 110864.

[7] Brackbill, J. U., Kothe, D. B., and Zemach, C. A continuum method for modeling surface tension. *Journal of computational physics 100*, 2 (1992), 335–354.

[8] Breiman, L., Friedman, J., Olshen, R., and Stone, C. Classification and regression trees. monterey, ca: wadsworth & brooks/cole advanced books & software, 1984.

[9] Colella, P., and Woodward, P. R. The piecewise parabolic method (ppm) for gas-dynamical simulations. *Journal of computational physics 54*, 1 (1984), 174–201.

[10] De Gennes, P.-G., Brochard-Wyart, F., and Quéré, D. *Capillarity and wetting phenomena: drops, bubbles, pearls, waves*. Springer Science & Business Media, 2013.

[11] Enright, D., Fedkiw, R., Ferziger, J., and Mitchell, I. A hybrid particle level set method for improved interface capturing. *Journal of Computational physics 183*, 1 (2002), 83–116.

[12] Gibou, F., Fedkiw, R. P., Cheng, L.-T., and Kang, M. A second-order-accurate symmetric discretization of the poisson equation on irregular domains. *Journal of Computational Physics 176*, 1 (2002), 205–227.

[13] Glimm, J., Isaacson, E., Marchesin, D., and McBryan, O. Front tracking for hyperbolic systems. *Advances in Applied Mathematics 2*, 1 (1981), 91–119.

[14] Godunov, S. Ph.d. dissertation: Different methods for shock waves moscow state university, 1954.

[15] Godunov, S. K., and Bohachevsky, I. Finite difference method for numerical computation of discontinuous solutions of the equations of fluid dynamics. *Matematičeskij sbornik 47*, 3 (1959), 271–306.

[16] Harten, A. High resolution schemes for hyperbolic conservation laws. *Journal of computational physics 135*, 2 (1997), 260–278.

[17] Hirt, C. W., and Nichols, B. D. Volume of fluid (vof) method for the dynamics of free boundaries. *Journal of computational physics 39*, 1 (1981), 201–225.

[18] Huang, Z., Lin, G., and Ardekani, A. M. Consistent, essentially conservative and balanced-force phase-field method to model incompressible two-phase flows. *Journal of Computational Physics 406* (2020), 109192.

[19] Jemison, M., Loch, E., Sussman, M., Shashkov, M., Arienti, M., Ohta, M., and Wang, Y. A coupled level set-moment of fluid method for incompressible two-phase flows. *Journal of Scientific Computing 54*, 2-3 (2013), 454–491.

[20] Jemison, M., Sussman, M., and Arienti, M. Compressible, multiphase semi-implicit method with moment of fluid interface representation. *Journal of Computational Physics 279* (2014), 182–217.

[21] Kasmaiee, S., Tadjfar, M., Kasmaiee, S., and Ahmadi, G. Linear stability analysis of surface waves of liquid jet injected in transverse gas flow with different angles: S. kasmaiee et al. *Theoretical and Computational Fluid Dynamics 38*, 1 (2024), 107–138.

[22] Krake, T., Klötzl, D., Eberhardt, B., and Weiskopf, D. Constrained dynamic mode decomposition. *IEEE Transactions on Visualization and Computer Graphics 29*, 1 (2023), 182–192.

[23] Lehoucq, R. B. Implicitly restarted arnoldi methods and subspace iteration. *SIAM Journal on Matrix Analysis and Applications 23*, 2 (2001), 551–562.

[24] Li, G., Lian, Y., Guo, Y., Jemison, M., Sussman, M., Helms, T., and Arienti, M. Incompressible multiphase flow and encapsulation simulations using the moment-of-fluid method. *International Journal for Numerical Methods in Fluids 79*, 9 (2015), 456–490.

[25] Liu, Y., Sussman, M., Lian, Y., Hussaini, M. Y., Vahab, M., and Shoele, K. A novel supermesh method for computing solutions to the multi-material stefan problem with complex deforming interfaces and microstructure. *Journal of Scientific Computing 91*, 1 (2022), 1–40.

[26] Markstein, G. Interaction of flow pulsations and flame propagation. *Journal of the Aeronautical Sciences 18*, 6 (1951), 428–429.

[27] Meehan, M. A., Simons-Wellin, S., and Hamlington, P. E. Efficient algorithm for proper orthogonal decomposition of block-structured adaptively refined numerical simulations. *Journal of Computational Physics 469* (2022), 111527.

[28] Milcent, T., and Lemoine, A. Moment-of-fluid analytic reconstruction on 3d rectangular hexahedrons. *Journal of Computational Physics 409* (2020), 109346.

[29] Olsson, E., and Kreiss, G. A conservative level set method for two phase flow. *Journal of computational physics 210*, 1 (2005), 225–246.

[30] Osher, S., and Sethian, J. A. Fronts propagating with curvature-dependent speed: algorithms based on hamilton-jacobi formulations. *Journal of computational physics 79*, 1 (1988), 12–49.

[31] Pei, C., Vahab, M., Sussman, M., and Hussaini, M. Y. A hierarchical space-time spectral element and moment-of-fluid method for improved capturing of vortical structures in incompressible multi-phase/multi-material flows. *Journal of Scientific Computing 81*, 3 (2019), 1527–1566.

[32] Qiu, R., Huang, R., Xiao, Y., Wang, J., Zhang, Z., Yue, J., Zeng, Z., and Wang, Y. Physics-informed neural networks for phase-field method in two-phase flow. *Physics of Fluids 34*, 5 (05 2022). 052109.

[33] Ranjan, R., Unnikrishnan, S., and Gaitonde, D. A robust approach for stability analysis of complex flows using high-order navier-stokes solvers. *Journal of Computational Physics 403* (2020), 109076.

[34] Salas, M. D. Shock fitting method for complicated two-dimensional supersonic flows. *AIAA Journal 14*, 5 (1976), 583–588.

[35] Saurel, R., and Abgrall, R. A multiphase godunov method for compressible multifluid and multiphase flows. *Journal of Computational Physics 150*, 2 (1999), 425–467.

[36] Schmid, P. J. Dynamic mode decomposition of numerical and experimental data. *Journal of fluid mechanics 656* (2010), 5–28.

[37] Shin, S., and Juric, D. A hybrid interface method for three-dimensional multiphase flows based on front tracking and level set techniques. *International Journal for Numerical Methods in Fluids 60*, 7 (2009), 753–778.

[38] Shu, C.-W., and Osher, S. Efficient implementation of essentially non-oscillatory shock-capturing schemes. *Journal of computational physics 77*, 2 (1988), 439–471.

[39] Sussman, M., and Ohta, M. A stable and efficient method for treating surface tension in incompressible two-phase flow. *SIAM Journal on Scientific Computing 31*, 4 (2009), 2447–2471.

[40] Sussman, M., and Puckett, E. G. A coupled level set and volume-of-fluid method for computing 3d and axisymmetric incompressible two-phase flows. *Journal of Computational Physics 162*, 2 (2000), 301–337.

[41] Sussman, M., Smereka, P., and Osher, S. A level set approach for computing solutions to incompressible two-phase flow. *Journal of Computational physics 114*, 1 (1994), 146–159.

[42] Tatebe, O. The multigrid preconditioned conjugate gradient method. In *NASA. Langley Research Center, The Sixth Copper Mountain Conference on Multigrid Methods, Part 2* (1993).

[43] Unverdi, S. O., and Tryggvason, G. A front-tracking method for viscous, incompressible, multi-fluid flows. *Journal of computational physics 100*, 1 (1992), 25–37.

[44] Vahab, M., Sussman, M., and Shoele, K. Fluid-structure interaction of thin flexible bodies in multi-material multi-phase systems. *Journal of Computational Physics 429* (2021), 110008.

[45] Van Leer, B. Towards the ultimate conservative difference scheme. v. a second-order sequel to godunov's method. *Journal of computational Physics 32*, 1 (1979), 101–136.